

Lineo Warp!! 5.1 User's Manual



Rev. 4.4 01/
06/2018



Rev.	date	What's changed
1.0	2009/08/11	first edition
1.1	2009/11/13	Adding Silent Mode Changing the exit code of a snapshot script 5.3 Warp!! Adding Errors on Startup
2.0	2010/06/08	Warp!! 3.0 compatible 3 Add a function description 5.2 Adding Snapshot Deletion Adding Separate Mode Shrink Mode Adding ENODEV Addition Correction of misspellings
2.1	2010/08/06	Format change Correction of typographical errors
2.2	2010/09/06	Warp!! 3.1 support Added high-speed compression mode Added save cancellation function Added command specification during separate mode 2 pass
2.3	2011/03/22	2 Target Person: Change of description 4.1 Snapshot Acquisition Description Changes 4.2 Error Judgment of Snapshot Acquisition Description Change 5.1 Warp!! Startup Description Changes 5.2 Deleting a snapshot and changing the description 6 Customization Add
2.4	2011/06/23	4.2 Error Determination for Snapshot Acquisition Error Addition for MIPS 6.4 Snapshot Image Hookup Description Added
2.5	2011/08/31	Warp!! Version changes
3.0	2012/10/31	Warp!! 4.0 compatible 3 Function description Description change 4.3 Snapshot script options Switch function added 4.4 /proc/warp/ Switch function added 4.5 Use Cases Add 6.1 Changing the storage area Description change 6.2 Changing the Hibernation Driver Area Description Changes 7 Restrictions Added
3.1	2013/05/22	4.4 Remove /proc/warp/ mode and add switch 6.4 Snapshot Image Sucking Up SD Card Description Added 6.5 SD/MMC Card Replacement Added
4.0	2015/04/16	Warp!! 5.0 Compatible 3 Function description Change area size 3.3 Halt Mode Description Changes 3.4 Multiple snapshot function Description changes 3.6 Compression Modes Added Modes 10~13 3.9 Floating Driver Description Added 3.11 MultiCore Function Added 4.3 Snapshot script options – v Add 4.4 Added /proc/warp/ division 4.5 Use Cases Add Description 5.3 Warp!! Added Error Descriptions on Startup 6.1 Device Designation Added 6.2 Changing the Hibernation Driver Area Description Changes 6.3 Changing the storage area Description change 6.4 Changing CPU Information Added 6.6 Copying a snapshot image Description change

		6.8 Application Support Added 6.9 Warp!! Post-startup behavior added 6.10 Speed Improvement Method Addition 7.1 Fast Startup Mode Description Changes
4.1	2015/07/30	6.2.2 Linux kernel (if stored in a floating driver/ MTD) Error correction 6.3.1 Storage Area Settings (.bootflag / .snapshot) Descriptions Added 6.6.2 Copying Restrictions Descriptions
4.2	2016/02/09	5.3 Warp!! Error at startup Correction
4.3	2016/07/26	6.2.5 Linux kernel (non-floating driver) Error correction
4.4	2018/01/06	Warp!! 5.1 Compatible 6.Customization NVMe Description Addition 6.2.5 Linux kernel (non-floating driver) variable size description added

caution

- The copyright of this book belongs to Lineo Solutions Co., Ltd.
- You may not use or reproduce part or all of this document without permission.
- Lineo Solutions Co., Ltd. does not make any warranties or representations regarding the results of the operation of this document.
- The specifications of the Software and the contents of this document are subject to change without notice.

trademark

Lineo Solutions, Inc. is a trademark of Lineo Solutions Inc. Linux is a registered trademark owned by Linus Torvalds. Other company and product names mentioned are trademarks and registered trademarks of their respective companies.

Part No.:14113-0A0LJ00

Copyright© 2008-2018 Lineo Solutions, Inc. All rights reserved.

PartNo. : 11-0100-001

PartNo. : 11-0100-001

PartNo. : 11-0100-001

table of contents

1. outline	7
2. Target Audience	7
3. Function Description	7
3.1 Boot Mode	8
3.2 OneShot Mode	8
3.3 Halt Mode	8
3.4 Multiple Snapshot Function	9
3.5 Switch Functions	9
3.6 Compression Mode	10
3.7 Shrink Mode	11
3.8 Separate Mode	12
3.9 Floating Driver	13
3.10 Save Cancel Function	13
3.11 MultiCore Features	14
4. How to use it on the Linux kernel	15
4.1 Taking a Snapshot	15
4.2 Error Determination of Snapshot Acquisition	15
4.3 Snapshot Script Options	16
4.4 /proc/warp/	17
4.5 Use Cases	18
5. How to use it on the bootloader	19
5.1 Warp!! Startup	19
5.2 Deleting a Snapshot	19
5.3 Warp!! Startup Error	19
6. Customization	20
6.1 Device Designation	20
6.2 Changing the Hibernation Driver Region	21
6.2.1 Bootloader	21
6.2.2 Linux kernel (if stored in a floating driver/ MTD)	21
6.2.3 Linux kernel (if stored in a floating driver/ ATA/NVMe/SD)	22
6.2.4 Linux kernel (for floating drivers/ MTD unmapped NOR, etc.)	22
6.2.5 Linux kernel (non-floating driver)	23
6.3 Changing the Storage Area	24
6.3.1 Save Area Settings (.bootflag / .snapshot)	26
6.3.2 Switch Function Settings (.sw_bootflag).....	27
6.4 Changing CPU Information	28
6.5 Changing the Auto-Launch Snapshot Number	28
6.6 Copying a Snapshot Image.....	29
6.6.1 Snapshot Image Information	29
6.6.2 Copying Constraints	30
6.7 Replacing the SD/MMC Card	30
6.8 Application Response	31
6.9 Warp!! Behavior after startup	31
6.10 Acceleration Techniques	32

6.10.1 Compression Mode	32
6.10.2 Clearing Memory	32
6.10.3 Hibernation Driver Call Position for Bootloader	32
6.10.4 Shrink Mode	32
6.10.5 Separate Mode Without Swap	33
7. Restrictions	34
7.1 Fast Startup Mode	34

1. outline

This document is an instruction manual on how to use Lineo Warp!! 5.1.

2. Target Audience

As a reader of this book, I would like to hear the implemented Warp!! Snapshot Save using / Warp!! It is intended for users who launch the snapshot and users who perform simple customizations such as changing the snapshot storage area.

3. function description

Warp!! is a fast start-up solution that utilizes hibernation technology. It stores memory contents/register values at any point in time as snapshot images on the secondary storage device and expands them all at once at startup to achieve fast startup.

Warp!! The following areas are required for secondary storage.

- Hibernation Driver Area

Warp!! This is the storage area of the hibernation driver, which is the main body of the It is usually about 32KB~128KB in size.

- Boot flag area

Warp!! It is a management area of It is written when the snapshot image is saved and stores information such as the save mode and save size. The size is 1024 bytes.

- Snapshot Area

This is the storage area for memory contents. The size varies depending on the amount of memory in use.

3.1 Boot Mode

You can choose between a fast boot mode that uses the same snapshot every time it boots up, and a general hibernation mode that saves a snapshot every time it is powered off.

- **Fast boot mode**

This mode takes a snapshot in advance in the initial boot state (home screen, etc.) and starts from the same snapshot each time. When the power is turned off, there is no need to take a snapshot, and the power can be turned off at any time.

If you want to use this mode, disable OneShot mode to take a snapshot. The use of this mode is **subject to the constraints described in 7.1 Fast Boot Mode**.

- **Hibernation mode**

This mode provides a common hibernation function that saves a snapshot every time the power is turned off. Snapshots taken are automatically erased once used. If you want to use this mode, enable OneShot mode/ Halt mode to take a snapshot.

3.2 OneShot Mode

This mode is intended for use as a general hibernation. If you specify this mode, Warp !! After booting, the boot flag area will be automatically cleared, and Warp will be able to save the snapshot again!! It will not be able to start.

3.3 Halt Mode

This mode does not return to the kernel after taking a snapshot and stops the system. After the save is finished, it will be an infinite loop in the hibernation driver.

If you specify OneShot mode, you will usually specify this as well. If you want to stop the operation on the Linux kernel side, you can not specify it. In this case, it must be stopped before access to the file system is made. If not, **the constraints described in the 7.1 fast boot mode** can cause the file system to break (for writable file systems). If an error occurs while saving a snapshot or the save is canceled, it will return to the kernel without stopping.

3.4 Multiple Snapshot Function

Warp!! supports multiple snapshot images. When saving a snapshot, specify the snapshot number to be saved. At startup, you can select the snapshot number to launch from the bootloader.

3.5 Switch Functions

Perform snapshot switching while the Linux kernel is running. Support for the multi-snapshot feature is required.

You can choose the following modes:

- **[Mode 0] Switch Disabled (default)**

I don't use the Switch function. Perform normal snapshot saving behavior.

- **[Mode 1] Overwrite Mode**

Discard the currently running environment and load the specified snapshot. This mode is mainly for snapshot switching while operating in fast boot mode (non-OneShot mode). This mode allows you to switch between multiple operating environments at high speed. However, after the switch, it will always be in the initial boot state.

Warp!! If you use this mode after booting, the snapshot image before switching to **7.1** You will be subject to the constraints described in Fast Boot Mode.

- **[Mode 2] Exchange Mode**

After saving a snapshot of the currently running environment, load the specified snapshot. This mode is mainly for snapshot switching while operating in hibernation mode (OneShot mode). When using this mode, you usually specify OneShot mode. If you do not specify it, **you will be subject to the constraints described in 7.1 Fast Boot Mode**. Since snapshot storage is required, the switch is slower than in Overwrite mode, but it is possible to return to the pre-switch environment by using the Switch function again. You can also use a combination of the two modes: Overwrite/Exchange. For example, if you want to temporarily migrate to a different operating environment (always the one used for initial startup) during normal operation, and then return to normal operation in the middle of operation, you can migrate to another operating environment in Exchange mode and return to the original environment in Overwrite mode.

3.6 Compression Modes

You can choose to compress/uncompress the snapshot image. The compression ratio varies depending on the data, but is usually less than half the size. Depending on the operating clock and the destination transfer rate, saving is generally faster than uncompressed and booting is generally faster. Note that the 2nd load part of the separate mode is always uncompressed.

You can choose the following modes:

- **[Mode 0] Uncompressed**

It does not perform compression. The storage time is fast, but the storage size is the largest. However, for devices with slow transfer rates, such as NOR Flash, the save time may be slower than compression.

- **[Mode 1] Normal Compression**

Do compression. The storage time is slow, but the storage size is the smallest. In general, this mode is the fastest to start.

- **[Mode 2] Fast Compression (Default)**

Perform fast compression. The save time is faster than in mode 1, but the save size is slightly larger. In addition, the boot time is slightly slower than that of mode 1 as the save size increases. Depending on the data, the compression process time is about half that of mode 1, and the save size is about 10% more than that of mode 1.

- **[Mode 10~13] UserAPI Compression**

Compression is performed using the compression process of the UserAPI driver. Specifies the UserAPI driver to be used by the mode number .

Mode 10: UserAPI Driver 0 Mode
11: UserAPI Driver 1 Mode 12:
UserAPI Driver 2 Mode 13 :
UserAPI Driver 3

3.7 Shrink Mode

Perform a memory reclamation process before saving the snapshot. This process reduces the snapshot save size and reduces the size of the snapshot, and Warp!! Faster boot times. However, disk cache and loaded application space are also subject to recovery, so Warp!! This will result in a reload after startup. In this case, you may be concerned about the staggering depending on the application.

You can choose the following modes:

- **[Mode 0] No memory recovery**

No memory recovery process is performed. The save size is the largest.

- **[Mode 1] Maximum memory recovery (default)**

Memory recovery is done as much as possible. The save size is the smallest.

- **[Mode 2] Partial memory recovery (small recovery)**

It is an intermediate mode between mode 0 and mode 1. It mainly recovers unused memory.

- **[Mode 3] Memory recovery only partially (large amount of recovery)**

It is an intermediate mode between mode 0 and mode 1. It mainly frees up unused/unreferenced memory.

The snapshot save size is related to the following:

Mode 0> Mode 2> Mode 3> Mode 1

*Some modes are not supported depending on the Linux kernel version. *When you pass in separate mode 1, it will always be in mode 1.

3.8 Separate Mode

This mode saves snapshots in two types: memory that is read at startup (1st load) and memory that is loaded at any time after startup (2nd load). The smaller the 1st load, the faster the startup time, but the more readings after startup, the more stuttering the operation and operation will occur for a while.

You can choose the following modes:

- **[Mode 0] Normal Mode (Default)**

This is a normal mode that does not use 2nd load and reads all memory with 1st load.

- **[Mode 1] Separate Mode 1pass**

This mode minimizes 1st load and has the fastest boot time. If this mode is specified, the shrink mode will be in mode 1 (maximum memory recovery).

- **[Mode 2] Separate Mode 2pass**

This mode has a slightly larger save size of the 1st load part than mode 1. This mode reduces the stuttering after startup by storing the memory that is used immediately after startup in the memory stored in the 2nd load part of the 1st load part. The larger the 1st load part, the slower the boot time than in mode 1.

In this mode, the save process is performed 2pass. By changing the latency between 2 passes, it is possible to adjust the amount of memory stored in the 1st load to some extent. Also, be careful not to perform any operations or launch applications during this waiting time, as this will affect the memory size stored in the 1st load section.

It is also possible to wait between 2passes with a specified command instead of time. By controlling swap-in by sending a signal to the application you want to load preferentially from this command, you can specify a certain amount of memory to be stored in the 1st load part. The 2nd load part cannot be compressed. When using compression, mode 1/2 requires more storage space than mode 0. In addition, mode 2 requires more storage space because there is overlapping memory stored in the 1st and 2nd load sections.

The 2nd load part is saved by the swap function of the Linux kernel. You must register the swap area in advance. The swap area will be used only for the 2nd load and will no longer be used as a normal swap (if not in OneShot mode).

*Depending on the Linux kernel version, mode 1/2 is not supported.

3.9 Floating Driver

The hibernation driver looks for free space in memory, copies itself, and operates in that area. This method does not require fixed memory for the hibernation driver to operate. If you do not use a floating driver, you will need a dedicated area on the SRAM or SDRAM where the hibernation driver resides.

Floating drivers don't require dedicated space, but Linux kernels is using full memory, or if fragmentation does not allow contiguous space to be secured, the hibernation driver may not be able to operate. Especially in hibernation mode, fragmentation issues are more likely to occur. If you expect to run out of memory, we recommend operating with a non-floating driver.

The contiguous area required for a floating driver to operate is similar to the area size required for a non-floating driver. See **6.2.5 Linux kernel (non-floating driver)** for this size.

If the hibernation driver supports both floating and non-floating drivers, the bootloader/ Linux kernel definition file specifies which format to use. For information on how to change this, **see Changing the 6.2 Hibernation Driver Area**.

3.10 Save Cancellation Function

If the unsave function is supported, you can stop saving the snapshot in the middle of it. This feature is mainly for use when in OneShot mode. For example, it can be used for:

- (1) Stop the screen output, etc. by the power-off operation (apparent power off).
- (2) Perform the snapshot saving process behind the scenes.
- (3) If the power-on operation is performed while saving the snapshot, perform the following:
Perform the save cancel function and return to the kernel (apparent power-on).
- (4) If the snapshot saving is completed successfully, perform the following: Software control powers off. Warp with the next power-on operation !! Launch.

This makes it possible to save without reminding the user of the storage time. However, if the battery dies during storage, the next boot will be Linux booting normally.

3.11 MultiCore Features

If the MultiCore feature is supported, you can split and save snapshots using multiple CPUs, allocate dedicated CPUs for the UserAPI, and more.

The split save function allows you to split the snapshot into multiple parts and have each CPU perform the compression and deployment process in parallel to speed up the save/start time. You can specify the device to which the split snapshot image is stored separately. By using separate devices, you can expect even faster speeds.

When storing multiple snapshot images on a single device, the hibernation driver performs exclusive processing to ensure that there are no access contentions for the devices. This support is only supported for ATA/SD/UserAPI drivers. Don't save multiple snapshots on other devices. Note that if you want to store multiple on one device, the storage device will need a fairly fast transfer rate. Slower devices may not have a splitting effect.

The split percentage is specified when you save the snapshot. Also, when saving and Warp!! At startup, the processing time for each CPU is displayed as follows:

```
CPU0: Xfer Time XXXXXXXX usec
CPU1: Xfer Time XXXXXXXX usec
```

Warp!! The maximum effect can be achieved by adjusting the split ratio so that the processing time of each CPU at startup is as close as possible.

The MultiCore function allows you to specify the CPU number you want to use . CPU numbers can be physical CPU numbers determined by hardware or logical CPU numbers defined by the Linux kernel. For information about defining logical CPU numbers, **see 6.4 Changing CPU Information**.

4. How to use it on Linux kernel

4.1 Taking a Snapshot

Warp!! In order to start, you need to take a snapshot beforehand. Warp!! The startup resumes operation from the time the snapshot is taken. To take a snapshot, run the script for the capture. Root privileges are usually required to run .

```
# snapshot
```

When the snapshot capture is finished, it returns to the caller with an exit code of 0. Also, Warp!! On startup, it returns to the caller with exit code 1.

4.2 Error Determination of Snapshot Acquisition

If the snapshot capture fails, an error code is returned in /proc/warp/error. The error code is in accordance with the Linux kernel error code and has the following (the returned value is negative below):

Error	Error content
EIO(5)	I/O errors. Mainly writing failures.
ENOMEM(12)	Insufficient memory. Insufficient space prevents hibernation driver from working
ENODEV(19)	Device not found (PCI only)
EINVAL(22)	Parameter error. The parameters passed to the hibernation driver are abnormal (different versions, etc.).
ENOSPC(28)	Insufficient capacity in the snapshot area.
ETIMEDOUT(110)	Timeout occurred. Mainly writing failures.
ECANCELED(125)	Save cancellation (other than MIPS architecture).
ECANCELED(158)	Save cancellation (MIPS architecture only).

If successful, 0 will be returned. Also, Warp!! If you return to the Linux kernel by startup , it will always return 0. If an error occurs in the Suspend process of the device, a kernel error code other than the above may be returned.

4.3 Snapshot Script Options

Snapshot scripts support the following options:

option	explanation
-sN	Specify the save snapshot number when supporting multiple snapshots (N=0 ~). Switch function Also used as a save number in Exchange mode.
-lN	Specify the snapshot number to be loaded when the Switch function is performed (N=0~).
-wN	Mode designation of the Switch function. N=0 : Switch disabled (normal snapshot save) N=1 : Overwrite mode (snapshot load only) N=2 : Exchange mode (snapshot save & load)
-cN	Compression specification for snapshot data. N When omitted, it is 1 (compression designation). N=0 : No compression N=1 : Normal compression N=2 : Fast compression
-rN	Shrink mode designation. N is 1 (maximum memory recovery) when omitted . N=0 : No memory recovery N=1 : Maximum memory recovery N=2 : Only partial memory recovery (small recovery) N=3 : Only partial memory recovery (large recovery)
-pN	Separate mode designation. N When omitted, it becomes 1 (1 pass). N=0 : Normal Mode N=1 : 1pass N=2 : 2pass
-dN	Specifies the delay time (wait time between 2 passes) in seconds during separate mode 2pass.
-tCMD	Specifies when executing a command instead of delay time during separate mode 2pass. If this is specified, the delay time will be 0.
-vN	Specify the MultiCore split storage percentage. Logical CPU number 0, separated by a comma, specifies the percentage of each CPU saved. CPUs with 0 are not used for storage. Be sure to use CPU0 for storage.
-oN	OneShot mode designation. N is 1 (OneShot) when omitted . N=0 : Normal Mode N=1 : OneShot mode
-aN	Halt mode designation. N If omitted, it will be 1 (no return). N=0 : Return to kernel N=1 : This setting is ignored when the Switch function does not return to the kernel.
-iN	Silent mode designation. Suppress the serial output of the hibernation driver. If N is omitted, it will be 3 (neither is displayed). N=0 : Output on both save and startup N=1 : Output on save/no output on startup N=2 : No output on save / Output on startup N=3 : No output on both save and startup
-h	Show help.

The snapshot script takes a snapshot by setting the specified option to /proc/warp/ and then doing the following:

```
# echo disk > /sys/power/state
```

After reverting, the /proc/warp/ setting is reverted. Also, if each option is not specified, the current /proc/warp/ setting will be followed.

4.4 /proc/warp/

/proc/warp/ has an entry as an input parameter for the hibernation driver and an entry as an output parameter.

The following are the input parameters: For more information, **see the same section in 4.3 Snapshot Script Options.**

item	explanation
saveno	Specify the save snapshot number when multiple snapshots are supported. Switch function Also used as a save number in Exchange mode. The default value is specified in CONFIG_PM_WARP_SAVENO.
loadno	Specify the snapshot number to be loaded when the Switch function. The default value is specified in CONFIG_PM_WARP_SAVENO.
switch	Mode designation of the Switch function. The default value is 0.
compress	Compression specification for snapshot data. The default value is specified in CONFIG_PM_WARP_COMPRESS.
shrink	Shrink mode designation. The default value is specified in CONFIG_PM_WARP_SHRINK.
separate	Separate mode designation. The default value is specified in CONFIG_PM_WARP_SEPARATE.
division	Specify the MultiCore split storage percentage.
oneshot	OneShot mode designation. The default value is specified in CONFIG_PM_WARP_ONESHOT.
halt	Halt mode designation. The default value is specified in CONFIG_PM_WARP_HALT.
silent	Silent mode designation. The default value is specified in CONFIG_PM_WARP_SILENT.

The following are the output parameters:

item	explanation
stat	Will it return due to the end of snapshot saving, or Warp!! Returns startup. 0 : Snapshot Saved 1 : Warp!! launch
error	Returns an error code for saving snapshots. 0 : Successful Termination Negative: Error
retry	Snapshot save, or Warp!! Returns the total number of read/write retries of data that occurred at startup .
canceled	Returns the canceled state of the snapshot save. 0 : Successful Termination 1 : Saved and canceled. It is also possible to cancel the save by writing 1 between 2passes in separate mode 2pass. Also, if you want to use the save cancellation function, write 0 when you start saving .

4.5 Use Cases

Below is an example of using a snapshot script. Note that all of these are examples when the setting by /proc/warp/ is " -w0 -o0 -a0 ".

- Save with default settings (/proc/warp/ settings).

```
# snapshot
```

- Snapshot number 0 saved in fast boot mode.

```
# snapshot - s0
```

- Snapshot number 2 saved in hibernation mode.

```
# snapshot - s2 - o -a
```

- Discard the current environment and load snapshot number 1 (Switch feature Overwrite mode).

```
# snapshot - l1 - w1
```

- Save the current environment to snapshot number 2 in OneShot mode and load snapshot number 3 (Switch feature mode).

```
# snapshot - s2 - l3 - w2 -o
```

- Save at CPU0 100%.

```
# snapshot - v1
```

- Split storage at CPU0 50% and CPU1 50%.

```
# snapshot - v1,1
```

- Split storage with CPU0 50% and CPU2 50%.

```
# snapshot - v1,0,1
```

- Split storage at CPU0 25% and CPU3 75%.

```
# snapshot - v1,,,3
```

- Split storage with CPU0 50%, CPU1 25%, and CPU2 25%.

```
# snapshot - v2,1,1
```

- Split storage with CPU0 25%, CPU1 25%, CPU2 25%, and CPU3 25%.

```
# snapshot - v25,25,25,25
```

5. How to use it on the bootloader

5.1 Warp!! launch

If the snapshot is saved, it will automatically Warp!! Startup takes place. Depending on the bootloader, Warp from the prompt !! It supports startup.

```
# warp
```

For SH-IPL

```
> s
```

If multiple snapshots are supported, you can specify the snapshot number (0~) as a parameter . When omitted, the value specified in the CONFIG_WARP_SAVENO is used.

5.2 Deleting Snapshots

Some bootloaders support removing snapshots from prompts (clearing boot flags).

```
# clear_bootf
```

If multiple snapshots are supported, you can specify the snapshot number (0~) as a parameter . When omitted, the value specified in the CONFIG_WARP_SAVENO is used.

5.3 Warp!! Error on startup

If the hibernation driver does not support the version of the stored boot flag, it will return to the caller with an exit code of -22 (EINVAL).

If an error other than the above occurs, such as an error during the lead, the boot flag is rewritten and rebooted. The boot flag regions that can be rewritten are as follows:

offset	size	Changed values
0x000	4 bytes	0x4F525245
0x3F8	4 bytes	Error number (see 4.2) (negative binary value)

If you reboot in this state, the bootloader will Warp !! It does not boot and normally starts the Linux kernel. If you have set to automatically take a snapshot in the middle of a normal startup, you will be able to perform a fast startup again from the next startup. You can also check the above areas in the bootloader or Linux kernel to control the behavior when an error occurs.

6. Customization

In this chapter, we will change the source of the bootloader/Linux kernel and use Warp!! Learn how to customize the The customization file depends on the bootloader and kernel version. For the file names of the definition files, see the release notes. Please note that we do not guarantee that this will work after the customizations described in this chapter.

6.1 Device Designation

The device that stores hibernation drivers/boot flags/snapshots is specified in 32-bit numbers. In this number, the following three items are specified for each bit.

bit23-16 : 1=NOR 2=NAND 3=ATA 4=SD 5=MEM 6=SPI Flash 7=NVMe 126=UserAPI 127=Extension Driver

bit15-8 : Device number (multi-device support only) bit7-0 : Partition number in ATA/SD

Macro WARP_DEV (dev, lun, pat) are available for definition. dev has NOR / NAND / ATA
Specify the device number for lun and the partition number for part.

You can also store all boot flag/snapshot regions and hibernation drivers in a single partition.
Please specify the offset/size so that the areas do not overlap.

The bootloader CONFIG_WARP_xxx to enable the device (xxx is NOR / NAND / ATA /SD/MEM/SPI/NVME/USERDRV/EXTDRV). Be sure to define the device specified in bit23-16.
When loading hibernation drivers/boot flags in the Linux kernel, specify the device in the Linux kernel (MTD number or /dev/ Specify the device name below).

6.2 Changing the Hibernation Driver Region

To change the storage space of the hibernation driver, you can only change the source of the bootloader/Linux kernel. No hibernation driver changes are required. Modify the WARP_DRV_INFO in each definition file. Usually, the definition is as follows.

6.2.1 Bootloader

```
#define WARP_DRV_INFO {
.mode
    = WARP_DRV_FIXED, .addr .drv.dev .drv.offsets .drv.size
                                = 0xf0000000,
                                = WARP_DEV(NAND, 0, 0), =
                                0x00500000, = 0x00040000, }
```

.mode : WARP_DRV_FLOATING (specify a floating driver) or
WARP_DRV_FIXED (specify a non-floating driver) .addr : load address (defined

only when a non-floating driver). drv.dev : Storage device (see section **6.1 for specification method**) .drv.offsets : ATA/NVMe/SD offset from the top of the partition (in 512 bytes) Other offsets from the top of the device (in bytes)

.drv.size : Storage space size (defined only in NAND/ bytes)

6.2.2 Linux kernel (if stored in a floating driver/ MTD)

```
#define WARP_DRV_INFO {
.mode .drv.floating.load
    = WARP_DRV_FLOATING, = WARP_LOAD_MTD_NO,
    .drv.floating.dev.part = 3, .drv.floating.offsets
    = 0x00000000, }, }
```

.mode: WARP_DRV_FLOATING .drv.floating.load: WARP_LOAD_MTD_NO or
WARP_LOAD_MTD_NAME .drv.floating.dev.part: partition number
(WARP_LOAD_MTD_NO time only) .drv.floating.dev.name : partition name (
WARP_LOAD_MTD_NAME time only) .drv.floating.offsets: Offset from the top of the
partition (in bytes) .drv.floating.size : Storage space size (NAND only/bytes)

.drv.floating.dev.part / .drv.floating.dev.name specifies one or the other (depending on the
.drv.floating.load configuration). If the kernel version is 2.6.19 or earlier,
WARP_LOAD_MTD_NAME cannot be specified.

In the case of NAND Flash, the first healthy block (not a bad block) found in the area specified
in the storage space size is the actual storage area. The read size of the hibernation driver is
written by the hibernation driver itself, and it is not read by the storage space size.

6.2.3 Linux kernel (if stored in floating driver/ ATA/NVMe/SD)

```
#define WARP_DRV_INFO { .mode  
.drv.floating.load  
                                = WARP_DRV_FLOATING, =  
                                WARP_LOAD_DEV,  
.drv.floating.dev.name = " mmcblk1p3, .drv.floating.offsets }, =  
0x00000000, ¥¥¥¥¥¥¥¥
```

.mode: WARP_DRV_FLATING .drv.floating.load: WARP_LOAD_DEV .drv.floating.dev.name: device name (sda2/mmcblk0p3, etc.) .drv.floating.offsets: Offset from the beginning of the partition (in 512 bytes) If the device name does not start with "/", then "/dev/" is appended to the top. If this device name does not exist, "/dev/block/" will be appended.

6.2.4 Linux kernel (for floating drivers/ NOR without MTD mapping , etc.)

```
#define WARP_DRV_INFO { .mode  
.drv.floating.load  
                                = WARP_DRV_FLOATING, =  
                                WARP_LOAD_MEM,  
.drv.floating.dev.addr = 0xf0000000, .drv.floating.offsets }, =  
0x00000000, ¥¥¥¥¥¥¥¥
```

.mode: WARP_DRV_FLATING .drv.floating.load: WARP_LOAD_MEM
.drv.floating.dev.addr: storage address .drv.floating.offsets: offset from storage address (in bytes)

6.2.5 Linux kernel (non-floating driver)

```
#define WARP_DRV_INFO
{ .mode .drv.fixed.phys
.drv.fixed.virt          = WARP_DRV_FIXED, =
.drv.fixed.size },      0xf0000000,      =
                        0xf1000000,      =
                        0x00100000, ¥¥¥¥¥¥¥¥
```

.mode : WARP_DRV_FIXED .drv.fixed.phys :
physical address .drv.fixed.virt : virtual address
.drv.fixed.size : area size When using
non-floating drivers, The workpiece area

used by the hibernation driver is reserved in the .drv.fixed.size above. The size required for this area depends on the device where the boot flag/snapshot is stored, etc., but is roughly the sum of the following:

- Hibernation driver body 128KB: 128KB: by each UserAPI
- Hibernation Driver Work driver: per UserAPI Depending on the
- UserAPI driver body driver : 1 erase block size:
- UserAPI Driver Work/Stack 128KB~32MB: 256KB~32MB
- NOR/NAND/SPI Flash
- ATA
- NVMe/SD:

The buffer size for ATA/NVMe/SD varies depending on how much space is available. The larger the buffer size, the faster the transfer time may be. However, the effect is not very great.

If you use multiple UserAPI drivers, calculate them by their total size. However, the non-floating driver format of the UserAPI driver uses a separate area and is therefore excluded from the total size.

For example, if the boot flag is NAND with an erase block size of 128KB and the snapshot is stored in SD, the minimum required size is the following size (without the User API):

128KB (main unit) + 128KB (work) + 128KB (NAND) + 256KB (SD) = 640KB

When running MultiCore, you will need to use CPU space for areas other than the hibernation driver and UserAPI driver body mentioned above. In the case of the above example, twice as much space other than 128KB (main body) is required, for a total of 1152KB.

If the WARP_USE_FREE_MEM is defined in the Linux kernel definition file, etc., the memory other than the hibernation driver body/ UserAPI driver body is the Linux kernel is reserved in the free space of the In this case, .drv.fixed.size is only required for these driver bodies.

.drv.fixed.virt specifies the virtual address of the Linux kernel running. During hibernation driver operation, it is mapped to a different address (usually the same address as the physical address).

The Linux kernel specifies the following three items for a single snapshot :

- `.sw_bootflag`

Boot flag loading information used when using Switch functions. If you don't want to use the Switch function, you can do it undefined.

- `.bootflag`

Specify the boot flag save area.

- `.snapshot[]`

Specify the snapshot storage area. Normally, only `snapshot[0]` is specified. `[1]` and later are used when using MultiCore split saves. The storage space specified in each array is used in order from the smallest logical CPU number. For example, for a three-split using CPU0/2/3, the following is used:

CPU0: `.snapshot[0]` CPU2: `.snapshot[1]` CPU3: `.snapshot[2]`

`.snapshot[]` after array number 1 is undefined (yes, you will specify the same storage space as the previous array (`.snapshot[]`, dev is set to 0 The place where

If one is undefined, use an array that precedes it). This is divided equally by the number of CPUs using the same storage space, and the offset/size of each CPU is automatically set. For example, if you divide 5 by CPU0 ~4 with only `[0]/[3]` specified, the area of `[0]` will be divided by CPU0~2 In 3 equal parts, the area of `[3]` will be divided into 2 equal parts by CPU3 ~4 . This automatic partitioning function is also available when the area size is specified as 0 (partition termination) in ATA/SD.

The bootloader only specifies the `.bootflag` in the above. The settings should be the same as the Linux kernel.

6.3.1 Storage Area Settings (.bootflag / .snapshot)

.bootflag and .snapshot each have a bootflag/snapshot storage area.

Supported devices vary for each hibernation driver. Please refer to the release notes for support details.

- For NOR/NAND/SPI Flash
.dev : WARP_DEV(XXX, 0, 0) (XXX is NOR / NAND / SPI) .offs : Offset from the top of the device (in bytes) .size : Storage space size (in bytes)

Note that the offset is not an offset from the beginning of the partition. Also, both offset/size must be placed at the device's Erase block boundary. In the case of NAND Flash, it is necessary to consider the bad block and secure a large size. For example, the boot flag is 1 KB, but you need to have at least 2 blocks of it. However, if you are using an alternative block method of NAND Flash drivers, you can use only one block.

- For ATA/NVMe/SD
.dev : WARP_DEV (XXX, lun, part) (XXX is ATA / SD / NVME) .offs : Offset from the beginning of the partition (in 512 bytes) .size : Storage space size (512 byte increments)

part specifies the leading partition as 1. If .size is specified as a value that exceeds 0 or the end of the partition, the size from the beginning of the storage area to the end of the partition is the storage area size.

If you specify 0 for part , the specification is for the entire device regardless of the partition table (mainly for specifying areas outside the partition). In this case, .offs specifies the offset address from the beginning of the device.

As for partition table formats, it supports the common MBR partition format and GPT (GUID Partition Table) partition format. For MBR format, the partition type should be 0xA0. You can also specify an extended partition.

For GPT format, the partition type GUID should be as follows, which is for Linux swap.

0657FD6D-A4AB-43C4-84E5-0933C84B4F4F

If you specify the partition type in the gdisk command, specify 8200 to get this GUID.

- For UserAPI
.dev : WARP_DEV(USER, lun, part) .offs : offset .size : storage space size

The lun specifies the UserAPI driver number to use. The method of specifying part / .offs / .size (units, etc.) is defined for each UserAPI driver.

6.3.2 Switch Function Settings (.sw_bootflag)

.sw_bootflag is a setting that allows the Linux kernel to load boot flags when the Switch function is functional. To load using the Linux kernel's device driver, specify the kernel-managed MTD partition or the device under /dev/. If you don't want to use the Switch function, you can do it undefined.

The destination where the boot flag is saved will change what you specify.

- For NOR/NAND (MTD partition numbering)
.load : WARP_LOAD_MTD_NO .dev.part : partition number .offs : MTD offset
from the beginning of the partition (in bytes)

The area size uses the value of .bootflag.size.

- For NOR/NAND (MTD partition name specified)
.load : WARP_LOAD_MTD_NAME .dev.name : partition name .offs : MTD offset from the
beginning of the partition (in bytes)

The area size uses the value of .bootflag.size.

If the kernel version is 2.6.19 or earlier, you cannot specify a partition name.

- For ATA/NVMe/SD
.load : WARP_LOAD_DEV .dev.name : device name (sda2 / mmcblk0p3,
etc.) .offs : offset from the beginning of the partition (in 512 bytes)

If the device name does not start with "/", it will be prefixed with "/dev/". If this device name does not exist, "/dev/block/" will be appended.

- For memory
.load : WARP_LOAD_MEM .offs : offset from memory top (in
bytes)

6.4 Changing CPU Information

If you want to change the physical CPU number used by the MultiCore feature, you change the WARP_CPU_INFO in the Linux kernel definition file . Usually, the definition is as follows.

```
#define WARP_CPU_INFO                                     ¥
{
    ¥.cpu
                                     = 0,
                                     ¥
    ¥}, { .save_ratio
                                     = 100,
    ¥.cpu
    ¥.save_ratio
                                     = 1,
                                     = 0,
                                     ¥
},
```

.cpu : Physical CPU Number

.save_ratio : Storage Ratio

The number of arrays specified here is the maximum number of CPUs available , and each array number is the logical CPU number.

The save ratio can be changed with the -v option in the snapshot script or in /proc/warp/division. Here we set the default values.

6.5 Changing the Auto-Start Snapshot Number

Warp!! If boot is auto-started, boot from the snapshot number specified in the CONFIG_WARP_SAVENO of the bootloader definition file. You can change this definition to change the snapshot number that auto-fires. Note that some bootloaders may be able to change the boot snapshot number with environment variables. In this case, the environment variable takes precedence.

6.6 Copying the Snapshot Image

In fast boot mode (a mode that uses the same snapshot every time at startup), the entire saved snapshot image can be copied to other devices for use. Even if you don't take a snapshot on the copied device, you can still use Warp!! It will be able to start.

6.6.1 Snapshot Image Information

Snapshot image information is available at the time of saving and Warp!! See the following that appears at startup.

```
Bootflag dev: (SD , 1, 3), offs: 0x0000000000000100 Snapshot dev: (SD , 1, 3), offs:  
0x0000000000000102 Snapshot total size: 0x15686680 compressed size:  
0x05b0ed94
```

Rows 1~2 show where the boot flag/snapshot was saved, respectively. The offset value indicated by the offs of the device indicated by dev: is the save position (see section 6.3.1 for the meaning of device/offset). Note that when saving a MultiCore split, multiple lines of the second line are displayed.

3 third line shows the byte size of the actual saved snapshot. When uncompressed, Snapshot total size, compressed size is the size to be copied when compressed .

The snapshot data and boot flag data are paired. Be sure to copy both and use them in a set. The boot flag data is 1 KB. When sucking in the Linux kernel, do the following (the specified number is the case in the above example):

```
# dd if=/dev/mmcblk1p3 of=bootflag.bin bs=512 skip=256 count=2 # dd if=/dev/mmcblk1p3  
of=snapshot.bin bs=512 skip=258 count=186487
```

6.6.2 Copying Restrictions

If you want to copy a snapshot image to another device, there are the following restrictions on the from/destination hardware:

- The hardware must be the same (including options).
- The file system being mounted (except for those on DRAM such as tmpfs) must be the same at the binary image level when taking a snapshot (tar It is not possible to write the file system by unfolding, etc.). If you want to rewrite a file before taking a snapshot, you need to suck up/copy the file system image after the snapshot is taken.
- All block device partitioning settings (number/position/size) recognized by the Linux kernel at the time of snapshot acquisition are identical.
- Device-specific settings such as network MAC address and touch panel calibration data can be used by Warp!! It must be reconfigured for each device after startup (kernel resume processing or reconfiguration on the application side).
- In the case of devices with bad blocks, such as NAND Flash, it is necessary to replace the difference in the position of the bad block between the two devices.
- When saving snapshots, if the file system on the SD/MMC card is mounted, the card information must be updated (**see 6.7 Replacing the SD/MMC Card**).

There are usually no restrictions on file systems that are not mounted at the time of snapshot acquisition, but caution should be exercised if files are temporarily mounted and rewritten before snapshot acquisition. Warp!! If the application needs to be rewritten after startup, it may not work properly without making a copy of this file system as well. In this case, it is generally not necessary to be identical at the image level, so it is okay to siphon/copy it with tar commands, etc. However, if you are storing the inode number of a file (such as SQLite on Android), the inode number of each file will be the same as when you took a snapshot. It must be copied at the image level. For systems with NAND Flash, the Linux kernel Warp!! The patch includes the bad block replacement of the mounted file system. You can enable this process by defining a WARP_MTD_BB_CONV in the customization definition file. However, the replacement will only be done if the NAND Flash was mounted via MTD. If you want to access NAND Flash for other reasons, you will need to take action individually. If you want to suck up an image from NAND Flash, you need to read it while skipping the bad block. We have a sample program nand_read.c to do this. For more information, see the samples directory in the delivery CD.

6.7 Replacing the SD/MMC Card

If you save a snapshot with the file system mounted on the SD/MMC card I/F, replacing the SD/MMC card with the file system written on it with another card will show that the card's CID Due to discrepancies in the serial number information in the register, Warp!! After booting, you may not be able to access the file system normally.

In this case, by defining the WARP_MMC_SNAPSHOT_COPY in the customization file of the Linux kernel , Warp!! The information in the CID register is updated after boot and can be accessed normally. Note that card information other than the CID register will work as the information of the old card, so if the old and new cards do not have the same type or partition configuration, they may not work properly.

6.8 Application Support

Warp!! uses the CONFIG_PM mechanism that the Linux kernel originally has. Therefore, if the device driver or application does not support this, the snapshot may not be taken properly.

For example, suspend / resume sometimes does the same thing as SIGSTOP / SIGCONT, but the process that does not accept these signals (TASK_UNINTERRUPTIBLE process) is running, snapshots cannot be taken.

In addition, when a system call such as read() / write() receives the above signal, it interrupts the processing and returns immediately, but in this case, the error code EINTR may return. In this case, the calling application will have to call the system call again without considering it an error, but the application that does not take this action may not work properly after taking the snapshot. Please be sure to respond to EINTR.

6.9 Warp!! Behavior after startup

Warp!! If you want to automatically start the application after startup, you can do so by writing the process that follows the snapshot script. For example, create another script that calls the snapshot script, such as: By using this script to take snapshots, you can use Warp!! You can specify what it will do after startup.

```
#!/bin/sh
snapshot "$@"
/usr/bin/user_application
```

Also, Warp!! After startup, it is usually necessary to reset the time from RTC or reconnect to the network, but it is effective to write it in this script.

6.10 Speed Improvement Techniques

In this section, we will discuss Warp !! Here are some methods for further acceleration after introduction.

6.10.1 Compression Modes

The default value for normal compression mode is mode 2. If saving time is not an issue, then using mode 1 is usually slightly faster to boot.

6.10.2 Clearing Memory

Warp!! Snapshot storage does not save unused memory, but since the unit of storage is page size units, some of the fractional unused space of Linux kernel memory will also be stored. If there is meaningless garbage in this area, it will reduce the compression ratio and affect the startup time.

When saving snapshots, there are cases where the compression ratio can be slightly increased and the speed can be increased by clearing the memory to zero with a bootloader and then starting Linux.

Also, if you specify an additional save area by `warp_set_savearea()` and only a part of the area is written at the time of saving the snapshot, you can expect a pre-zero clear effect on this area.

6.10.3 Bootloader Hibernation Driver Call Position

Warp!! At startup, the hibernation driver is called from the bootloader, and moving this call position forward can be expected to speed up to some extent. Especially if you have a device initialization that takes a long time, it can be helpful to move it before that initialization.

However, hibernation drivers/boot flags are loaded using the bootloader's driver, so they cannot be moved before the driver's initialization process. Also, if the Linux kernel's resume process assumes device initialization by the bootloader, the move of the call position may cause the device's resume to not work properly. The call position must be determined in relation to the resume processing of the device.

6.10.4 Shrink Mode

The default value of Shrink mode is usually 1, which is designed to maximize memory recovery. In this mode, the snapshot size is the smallest and Warp!! Snapshot load times are the fastest at startup, but shrinking frees up the application's .text space, disk cache, and more. Warp!! It does not affect operation because it is reloaded when the space is used after startup, but considering the overall boot time, it may be faster to change the shrink mode and avoid memory recovery. In addition, when initializing a device using the UserAPI, the following methods can be considered.

- If the snapshot load time is shorter than the device initialization time via the UserAPI, adjust the shrink mode to increase the snapshot size to make good use of the device initialization time.
- By using the UserAPI as much as possible to handle the resume of the Linux kernel, the increase in snapshot load time by adjusting the shrink mode is effectively utilized.

6.10.5 Separate Mode Without Swapping

Separate mode usually requires a swap, but you may be able to speed it up by using separate mode when swap does not exist.

If you run Mode 2 in separate mode without swapping , the following will happen:

- 1) Maximum memory reclamation execution.
- 2) Temporarily return to normal operation and wait for a specified time (or execute a specified command).
- 3) Perform memory reclamation in specified shrink mode.
- 4) Snapshot Acquisition.

Even if shrink mode 0 (no memory reclamation) is specified, 1) always performs maximum shrinking. After that, there is a reload of some of the areas that were freed by returning to normal operation in 2). The memory read here is Warp!! We can say that it is most likely the same as the memory that is accessed immediately after booting. If shrink mode 0/2/3 is specified3) does not perform complete memory recovery, these Warp!! The memory that is accessed immediately after boot is saved as a snapshot. This reduces the increase in snapshot size due to shrink mode changes, and also allows you to increase the size of the snapshot by changing the shrink mode, and further increase the number of snapshots that can be used in the Warp!! It can be expected to reduce reloads after startup.

7. Restrictions

7.1 Fast Startup Mode

When using fast boot mode, there are the following restrictions:

- Swap must not be used.
- The file system mounted at the time of snapshot acquisition (except for those on RAM) must be read-only (if the read-write file system is mounted, it must be unmounted before the snapshot is taken).
- Do not switch the above file system to readwrite after saving the snapshot (the entire partition cannot be rewritten at the binary image level).
- In the case of centrally managing multiple partitions, such as UBI, the above read-only constraints will be applied to the whole rather than the partition unit.

For the root file system, it is always safer to make it read-only, such as specifying ro on the kernel command line .

If you use UBI, it must be read-only, in units of ubiattach. For example, you can make the entire MTD partition read-only by configuring the following settings for the ubiattach (the configuration method depends on the kernel version).

```
static struct mtd_partition partition_info[] =
{
    { : :

        .mask_flags = MTD_WRITEABLE, },

};
```

As for the partition to read, it is divided into separate UBIs, and Warp!! Make sure to ubiattach after startup . If you are ubiattaching a read-right UBI at the time of snapshot acquisition, you need ubidetach before snapshot acquisition .

The above file system constraints are intended to ensure that the integrity of the file system contents and the file system information stored on RAM is not disrupted. When the file system is rewritten, the information on the RAM is usually updated at the same time, but when the fast boot mode is restarted in that state, only the information on the RAM will return to the one at the time the snapshot was taken. In this case, it may be inconsistent with the contents of the FIS system, and files may not be accessed normally, or newly created files may disappear.

Not only in fast boot mode, but all cases where the same snapshot image is loaded multiple times (including loading by the Switch function) will be subject to this restriction. In addition, if you disable Halt mode and perform a snapshot save, the file system may be rewritten when the kernel returns after the save is completed, so even OneShot mode is subject to restrictions.